

Pointer Variables

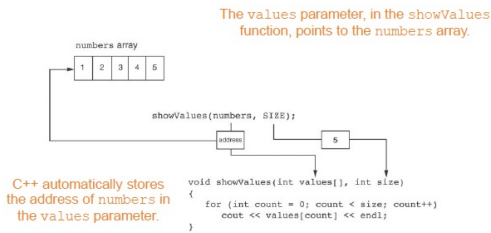
- **Pointer variable** : Often just called a pointer, it's a variable that holds an address
- Because a pointer variable holds the address of another piece of data, it "points" to the data

Something Like Pointers: Arrays

- We have already worked with something similar to pointers, when we learned to pass arrays as arguments to functions.
- For example, suppose we use this statement to pass the array `numbers` to the `showValues` function:

```
showValues(numbers, SIZE);
```

Something Like Pointers : Arrays



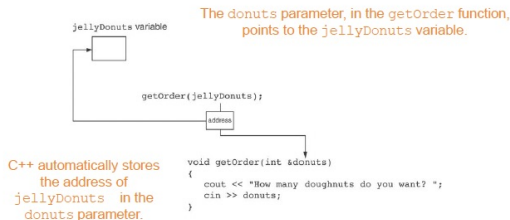
Something Like Pointers: Reference Variables

- We have also worked with something like pointers when we learned to use reference variables. Suppose we have this function:

```
void getOrder(int &donuts)
{
    cout << "How many doughnuts do you want? ";
    cin >> donuts;
}
```

- And we call it with this code:
- ```
int jellyDonuts;
getOrder(jellyDonuts);
```

## Something Like Pointers: Reference Variables



## Pointer Variables

- Pointer variables are yet another way using a memory address to work with a piece of data.
- Pointers are more "low-level" than arrays and reference variables.
- This means you are responsible for finding the address you want to store in the pointer and correctly using it.

## Pointer Variables

- **Definition:**  

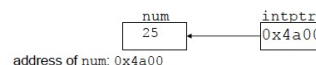
```
int *intptr;
```
- **Read as:**  
"intptr can hold the address of an int"
- **Spacing in definition does not matter:**  

```
int * intptr; // same as above
int* intptr; // same as above
```

## Pointer Variables

- Assigning an address to a pointer variable:  

```
int *intptr;
intptr = #
```
- Memory layout:



## Pointer Variables

- Initialize pointer variables with the special value `nullptr`.
- In C++ 11, the `nullptr` key word was introduced to represent the address 0.
- Here is an example of how you define a pointer variable and initialize it with the value `nullptr`:

```
int *ptr = nullptr;
```

## The Indirection Operator

- The indirection operator (\*) dereferences a pointer.
- It allows you to access the item that the pointer points to.

```
int x = 25;
int *intptr = &x;
cout << *intptr << endl;
```

This prints 25.

## The Relationship Between Arrays and Pointers

- Array name is starting address of array

```
int vals[] = {4, 7, 11};
```

|   |   |    |
|---|---|----|
| 4 | 7 | 11 |
|---|---|----|

starting address of vals: 0x4a00

```
cout << vals; // displays
 // 0x4a00
cout << vals[0]; // displays 4
```

## The Relationship Between Arrays and Pointers

- Array name can be used as a pointer constant:

```
int vals[] = {4, 7, 11};
cout << *vals; // displays 4
```

- Pointer can be used as an array name:

```
int *valptr = vals;
cout << valptr[1]; // displays 7
```

## Pointers in Expressions

Given:

```
int vals[]={4,7,11}, *valptr;
valptr = vals;
```

What is `valptr + 1`? It means (address in `valptr`) + (1 \* size of an int)

```
cout << *(valptr+1); //displays 7
cout << *(valptr+2); //displays 11
```

Must use ( ) as shown in the expressions

## Array Access

- Array elements can be accessed in many ways:

| Array access method                       | Example                          |
|-------------------------------------------|----------------------------------|
| array name and []                         | <code>vals[2] = 17;</code>       |
| pointer to array and []                   | <code>valptr[2] = 17;</code>     |
| array name and subscript arithmetic       | <code>*(vals + 2) = 17;</code>   |
| pointer to array and subscript arithmetic | <code>*(valptr + 2) = 17;</code> |

## Array Access

- Conversion: `vals[i]` is equivalent to `*(vals + i)`
- No bounds checking performed on array access, whether using array name or a pointer

## From Program 9-7

```
9 const int NUM_COINS = 5;
10 double coins[NUM_COINS] = {0.05, 0.1, 0.25, 0.5, 1.0};
11 double *doublePtr; // Pointer to a double
12 int count; // Array index
13
14 // Assign the address of the coins array to doublePtr.
15 doublePtr = coins;
16
17 // Display the contents of the coins array. Use subscripts
18 // with the pointer!
19 cout << "Here are the values in the coins array:\n";
20 for (count = 0; count < NUM_COINS; count++)
21 cout << doublePtr[count] << " ";
22
23 // Display the contents of the array again, but this time
24 // use pointer notation with the array name!
25 cout << "\nAnd here they are again:\n";
26 for (count = 0; count < NUM_COINS; count++)
27 cout << *(coins + count) << " ";
28 cout << endl;
```

### Program Output

```
Here are the values in the coins array:
0.05 0.1 0.25 0.5 1
And here they are again:
0.05 0.1 0.25 0.5 1
```

# Pointer Arithmetic

- Operations on pointer variables:

| Operation                | Example                                                                                            |
|--------------------------|----------------------------------------------------------------------------------------------------|
|                          | <pre>int vals[]={4,7,11}; int *valptr = vals;</pre>                                                |
| ++, --                   | <pre>valptr++; // points at 7 valptr--; // now points at 4</pre>                                   |
| +, - (pointer and int)   | <pre>cout &lt;&lt; *(valptr + 2); // 11</pre>                                                      |
| +=, -= (pointer and int) | <pre>valptr = vals; // points at 4 valptr += 2; // points at 11</pre>                              |
| - (pointer from pointer) | <pre>cout &lt;&lt; valptr - val; // difference // (number of ints) between valptr // and val</pre> |

## From Program 9-9

```

7 const int SIZE = 6;
8 int set[SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
9 int *numPtr = nullptr; // Pointer
10 int count; // Counter variable for loops
11
12 // Make numPtr point to the set array.
13 numPtr = set;
14
15 // Use the pointer to display the array contents.
16 cout << "The numbers in set are:\n";
17 for (count = 0; count < SIZE; count++)
18 {
19 cout << *numPtr << " ";
20 numPtr++;
21 }
22
23 // Display the array contents in reverse order.
24 cout << "The numbers in set backward are:\n";
25 for (count = 0; count < SIZE; count++)
26 {
27 numPtr--;
28 cout << *numPtr << " ";
29 }
30 return 0;
31 }
```

### Program Output

```

The numbers in set are:
5 10 15 20 25 30 35 40
The numbers in set backward are:
40 35 30 25 20 15 10 5
```

## Comparing Pointers

- Relational operators (<, >=, etc.) can be used to compare addresses in pointers
- Comparing addresses in pointers is not the same as comparing contents pointed at by pointers:

```

if (ptr1 == ptr2) // compares
 // addresses
if (*ptr1 == *ptr2) // compares
 // contents
```

## Pointers as Function Parameters

- A pointer can be a parameter
- Works like reference variable to allow change to argument from within function
- Requires:
  - asterisk \* on parameter in prototype and heading
  - asterisk \* in body to dereference the pointer
  - cin >> \*ptr;
  - address as argument to the function

```
getNum(&num); // pass address of num to getNum
```

## Example

```

void swap(int *x, int *y)
{
 int temp;
 temp = *x;
 *x = *y;
 *y = temp;
}

int num1 = 2, num2 = -3;
swap(&num1, &num2);
```

## Pointers as Function Parameters in Program 9-11

### Program 9-11

```

1 // This program uses two functions that accept addresses of
2 // variables as arguments.
3 #include <iostream>
4 using namespace std;
5
6 // Function prototypes
7 void getNumber(int *);
8 void doubleValue(int *);
9
10 int main()
11 {
12 int number;
13
14 // Call getNumber and pass the address of number.
15 getNumber(&number);
16
17 // Call doubleValue and pass the address of number.
18 doubleValue(&number);
19
20 // Display the value in number.
21 cout << "That value doubled is " << number << endl;
22 return 0;
23 }
24
```

(Program Continues)

## Pointers as Function Parameters in Program 9-11

### Program 9-11 (continued)

```

25 //*****
26 // Definition of getNumber. The parameter, input, is a pointer. *
27 // This function asks the user for a number. The value entered *
28 // is stored in the variable pointed to by input.
29 //*****
30
31 void getNumber(int *input)
32 {
33 cout << "Enter an integer number: ";
34 cin >> *input;
35 }
36
37 //*****
38 // Definition of doubleValue. The parameter, val, is a pointer. *
39 // This function multiplies the variable pointed to by val by *
40 // two.
41 //*****
42
43 void doubleValue(int *val)
44 {
45 *val *= 2;
46 }
```

### Program Output with Example Input Shown in Bold

```

Enter an integer number: 10 [Enter]
That value doubled is 20
```

## Pointers to Constants

- If we want to store the address of a constant in a pointer, then we need to store it in a pointer-to-const.

## Pointers to Constants

- Example: Suppose we have the following definitions:

```
const int SIZE = 6;
const double payRates[SIZE] =
 { 18.55, 17.45, 12.85,
 14.97, 10.35, 18.89 };
```

- In this code, `payRates` is an array of constant doubles.

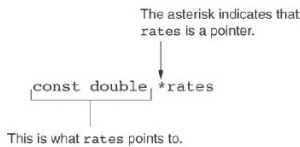
## Pointers to Constants

- Suppose we wish to pass the `payRates` array to a function? Here's an example of how we can do it.

```
void displayPayRates(const double *rates, int size)
{
 for (int count = 0; count < size; count++)
 {
 cout << "Pay rate for employee " << (count + 1)
 << " is $" << *(rates + count) << endl;
 }
}
```

The parameter, `rates`, is a pointer to `const double`.

## Declaration of a Pointer to Constant



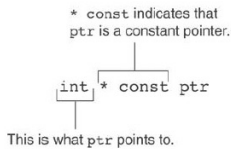
## Constant Pointers

- A constant pointer is a pointer that is initialized with an address, and cannot point to anything else.

- Example

```
int value = 22;
int * const ptr = &value;
```

## Constant Pointers



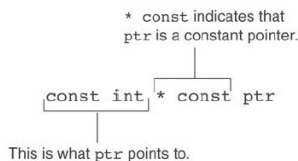
## Constant Pointers to Constants

- A constant pointer to a constant is:
  - a pointer that points to a constant
  - a pointer that cannot point to anything except what it is pointing to

- Example:

```
int value = 22;
const int * const ptr = &value;
```

## Constant Pointers to Constants



## Dynamic Memory Allocation

- Can allocate storage for a variable while program is running
- Computer returns address of newly allocated variable
- Uses `new` operator to allocate memory:

```
double *dptr = nullptr;
dptr = new double;
```
- `new` returns address of memory location



# Dynamic Memory Allocation

- Can also use `new` to allocate array:  
`const int SIZE = 25;`  
`arrayPtr = new double[SIZE];`
- Can then use `[]` or pointer arithmetic to access array:  
`for(i = 0; i < SIZE; i++)`  
`*arrayPtr[i] = i * i;`  
or  
`for(i = 0; i < SIZE; i++)`  
`*(arrayPtr + i) = i * i;`
- Program will terminate if not enough memory available to allocate

# Releasing Dynamic Memory

- Use `delete` to free dynamic memory:  
`delete fptr;`
- Use `[]` to free dynamic array:  
`delete [] arrayPtr;`
- Only use `delete` with dynamic memory!

## Dynamic Memory Allocation in Program 9-14

### Program 9-14

```
1 // This program totals and averages the sales figures for any
2 // number of days. The figures are stored in a dynamically
3 // allocated array.
4 #include <iostream>
5 #include <iomanip>
6 using namespace std;
7
8 int main()
9 {
10 double *sales = nullptr; // To dynamically allocate an array
11 total = 0.0; // Accumulator
12 average; // To hold average sales
13 int numDays; // To hold the number of days of sales
14 count; // Counter variable
15
16 // Get the number of days of sales.
17 cout << "How many days of sales figures do you wish ";
18 cout << "to process? ";
19 cin >> numDays;
```

## Dynamic Memory Allocation in Program 9-14

```
20
21 // Dynamically allocate an array large enough to hold
22 // that many days of sales amounts.
23 sales = new double[numDays];
24
25 // Get the sales figures for each day.
26 cout << "Enter the sales figures below.\n";
27 for (count = 0; count < numDays; count++)
28 {
29 cout << "Day " << (count + 1) << ": ";
30 cin >> sales[count];
31 }
32
33 // Calculate the total sales
34 for (count = 0; count < numDays; count++)
35 {
36 total += sales[count];
37 }
38
39 // Calculate the average sales per day
40 average = total / numDays;
41
42 // Display the results
43 cout << fixed << showpoint << setprecision(2);
44 cout << "\nTotal Sales: $" << total << endl;
45 cout << "Average Sales: $" << average << endl;
```

Program 9-14 (Continued)

## Dynamic Memory Allocation in Program 9-14

### Program 9-14 (Continued)

```
46
47 // Free dynamically allocated memory
48 delete [] sales;
49 sales = nullptr; // Make sales a null pointer.
50
51 return 0;
52 }
```

#### Program Output with Example Input Shown in Bold

```
How many days of sales figures do you wish to process? 5 [Enter]
Enter the sales figures below.
Day 1: 898.63 [Enter]
Day 2: 652.12 [Enter]
Day 3: 741.85 [Enter]
Day 4: 852.96 [Enter]
Day 5: 921.37 [Enter]
Total Sales: $4667.13
Average Sales: $933.43
```

*Notice that in line 49 `nullptr` is assigned to the `sales` pointer. The `delete` operator is designed to have no effect when used on a null pointer.*

## Returning Pointers from Functions

- Pointer can be the return type of a function:  
`int* newNum();`
- The function must not return a pointer to a local variable in the function.
- A function should only return a pointer:
  - to data that was passed to the function as an argument, or
  - to dynamically allocated memory

## From Program 9-15

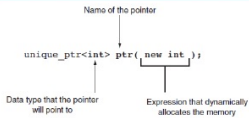
```
34 int *getRandomNumbers(int num)
35 {
36 int *arr = nullptr; // Array to hold the numbers
37
38 // Return a null pointer if num is zero or negative.
39 if (num <= 0)
40 return nullptr;
41
42 // Dynamically allocate the array.
43 arr = new int[num];
44
45 // Seed the random number generator by passing
46 // the return value of time(0) to srand.
47 srand(time(0));
48
49 // Populate the array with random numbers.
50 for (int count = 0; count < num; count++)
51 arr[count] = rand();
52
53 // Return a pointer to the array.
54 return arr;
55 }
```

## Using Smart Pointers to Avoid Memory Leaks

- In C++ 11, you can use *smart pointers* to dynamically allocate memory and not worry about deleting the memory when you are finished using it.
- Three types of smart pointer:
  - `unique_ptr`
  - `shared_ptr`
  - `weak_ptr`
- Must `#include` the memory header file:  
`#include <memory>`
- In this book, we introduce `unique_ptr`:  
`unique_ptr<int> ptr( new int );`

# Using Smart Pointers to Avoid Memory Leaks

Figure 9-12



- The notation `<int>` indicates that the pointer can point to an `int`.
- The name of the pointer is `ptr`.
- The expression `new int` allocates a chunk of memory to hold an `int`.
- The address of the chunk of memory will be assigned to `ptr`.

## Using Smart Pointers in Program 9-17

Program 9-17

```
1 // This program demonstrates a unique_ptr.
2 #include <iostream>
3 #include <memory>
4 using namespace std;
5
6 int main()
7 {
8 // Define a unique_ptr smart pointer, pointing
9 // to a dynamically allocated int.
10 unique_ptr<int> ptr(new int);
11
12 // Assign 99 to the dynamically allocated int.
13 *ptr = 99;
14
15 // Display the value of the dynamically allocated int.
16 cout << *ptr << endl;
17 return 0;
18 }
```

Program Output

99

## Abstract Data Types

- A data type that specifies
  - values that can be stored
  - operations that can be done on the values
- User of an abstract data type does not need to know the implementation of the data type, e.g., how the data is stored
- ADTs are created by programmers

## Abstraction and Data Types

- **Abstraction**: a definition that captures general characteristics without details
  - Ex: An abstract triangle is a 3-sided polygon. A specific triangle may be scalene, isosceles, or equilateral
- **Data Type** defines the values that can be stored in a variable and the operations that can be performed on it

## Combining Data into Structures

- **Structure**: C++ construct that allows multiple variables to be grouped together
- General Format:

```
struct <structName>
{
 type1 field1;
 type2 field2;
 . . .
};
```

## Example struct Declaration

```
struct Student ← structure tag
{
 int studentID; ←
 string name; ←
 short yearInSchool; ←
 double gpa; ←
};
```

structure members

## struct Declaration Notes

- Must have `;` after closing `}`
- `struct` names commonly begin with uppercase letter
- Multiple fields of same type can be in comma-separated list:

```
string name,
address;
```

## Defining Variables

- `struct` declaration does not allocate memory or create variables
- To define variables, use structure tag as type name:

Student bill;

|                                   |
|-----------------------------------|
| bill                              |
| studentID <input type="text"/>    |
| name <input type="text"/>         |
| yearInSchool <input type="text"/> |
| gpa <input type="text"/>          |

# Accessing Structure Members

- Use the dot (.) operator to refer to members of struct variables:

```
cin >> stu1.studentID;
getline(cin, stu1.name);
stu1.gpa = 3.75;
```

- Member variables can be used in any manner appropriate for their data type

## Program 11-1

```
1 // This program demonstrates the use of structures.
2 #include <iostream>
3 #include <string>
4 #include <omanip>
5 using namespace std;
6
7 struct PayRoll
8 {
9 int empNumber; // Employee number
10 string name; // Employee's name
11 double hours; // Hours worked
12 double payRate; // Hourly payRate
13 double grossPay; // Gross pay
14 };
15
16 int main()
17 {
18 PayRoll employee; // employee is a PayRoll structure.
19
20 // Get the employee's number.
21 cout << "Enter the employee's number: ";
22 cin >> employee.empNumber;
23
24 // Get the employee's name.
25 cout << "Enter the employee's name: ";
```

```
26 cin.ignore(); // To skip the remaining '\n' character
27 getline(cin, employee.name);
28
29 // Get the hours worked by the employee.
30 cout << "How many hours did the employee work? ";
31 cin >> employee.hours;
32
33 // Get the employee's hourly pay rate.
34 cout << "What is the employee's hourly payRate? ";
35 cin >> employee.payRate;
36
37 // Calculate the employee's gross pay.
38 employee.grossPay = employee.hours * employee.payRate;
39
40 // Display the employee data.
41 cout << "Here is the employee's payroll data:\n";
42 cout << "Name: " << employee.name << endl;
43 cout << "Number: " << employee.empNumber << endl;
44 cout << "Hours worked: " << employee.hours << endl;
45 cout << "Hourly payRate: " << employee.payRate << endl;
46 cout << fixed << showpoint << setprecision(2);
47 cout << "Gross Pay: $" << employee.grossPay << endl;
48 return 0;
49 }
```

## Program Output with Example Input Shown in Bold

```
Enter the employee's number: 489 [Enter]
Enter the employee's name: Jill Smith [Enter]
How many hours did the employee work? 40 [Enter]
What is the employee's hourly pay rate? 20 [Enter]
Here is the employee's payroll data:
Name: Jill Smith
Number: 489
Hours worked: 40
Hourly pay rate: 20
Gross pay: $800.00
```

## Displaying a struct Variable

- To display the contents of a struct variable, must display each field separately, using the dot operator:

```
cout << bill; // won't work
cout << bill.studentID << endl;
cout << bill.name << endl;
cout << bill.yearInSchool;
cout << " " << bill.gpa;
```

## Comparing struct Variables

- Cannot compare struct variables directly:

```
if (bill == william) // won't work
```

- Instead, must compare on a field basis:

```
if (bill.studentID ==
 william.studentID) ...
```

## Initializing a Structure

- struct variable can be initialized when defined:

```
Student s = {11465, "Joan", 2, 3.75};
```

- Can also be initialized member-by-member after definition:

```
s.name = "Joan";
s.gpa = 3.75;
```

## More on Initializing a Structure

- May initialize only some members:

```
Student bill = {14579};
```

- Cannot skip over members:

```
Student s = {1234, "John", ,
 2.83}; // illegal
```

- Cannot initialize in the structure declaration, since this does not allocate memory

## Excerpts From Program 11-3

```
8 struct EmployeePay
9 {
10 string name; // Employee name
11 int empNum; // Employee number
12 double payRate; // Hourly pay rate
13 double hours; // Hours worked
14 double grossPay; // Gross pay
15 };

19 EmployeePay employee1 = {"Betty Ross", 141, 18.75};
20 EmployeePay employee2 = {"Jill Sandburg", 142, 17.50};
```

### Program 11-4

```
1 // This program uses an array of structures.
2 #include <iostream>
3 #include <iomanip>
4 using namespace std;
5
6 struct PayInfo
7 {
8 int hours; // Hours worked
9 double payRate; // Hourly pay rate
10 };
11
12 int main()
13 {
14 const int NUM_WORKERS = 3; // Number of workers
15 PayInfo workers[NUM_WORKERS]; // Array of structures
16 int index; // Loop counter
17 }
```

### Program Output with Example Input Shown in Bold

```
Enter the hours worked by 3 employees and their hourly rates.
Hours worked by employee #1: 10 [Enter]
Hourly pay rate for employee #1: 9.75 [Enter]

Hours worked by employee #2: 20 [Enter]
Hourly pay rate for employee #2: 10.00 [Enter]

Hours worked by employee #3: 40 [Enter]
Hourly pay rate for employee #3: 20.00 [Enter]

Here is the gross pay for each employee:
Employee #1: $97.50
Employee #2: $200.00
Employee #3: $800.00
```

## Members of Nested Structures

- Use the dot operator multiple times to refer to fields of nested structures:

```
Student s;
s.pData.name = "Joanne";
s.pData.city = "Tulsa";
```

## Arrays of Structures

- Structures can be defined in arrays
- Can be used in place of parallel arrays

```
const int NUM_STUDENTS = 20;
Student stuList[NUM_STUDENTS];
```
- Individual structures accessible using subscript notation
- Fields within structures accessible using dot notation:

```
cout << stuList[5].studentID;
```

```
18 // Get employee pay data.
19 cout << "Enter the hours worked by " << NUM_WORKERS
20 << " employees and their hourly rates.\n";
21
22 for (index = 0; index < NUM_WORKERS; index++)
23 {
24 // Get the hours worked by an employee.
25 cout << "Hours worked by employee #" << (index + 1);
26 cout << ": ";
27 cin >> workers[index].hours;
28
29 // Get the employee's hourly pay rate.
30 cout << "Hourly pay rate for employee #";
31 cout << (index + 1) << ": ";
32 cin >> workers[index].payRate;
33 cout << endl;
34 }
35
36 // Display each employee's gross pay.
37 cout << "Here is the gross pay for each employee:\n";
38 cout << fixed << showpoint << setprecision(2);
39 for (index = 0; index < NUM_WORKERS; index++)
40 {
41 double gross;
42 gross = workers[index].hours * workers[index].payRate;
43 cout << "Employee #" << (index + 1);
44 cout << ": $" << gross << endl;
45 }
46 return 0;
47 }
```

## Nested Structures

A structure can contain another structure as a member:

```
struct PersonInfo
{
 string name,
 address,
 city;
};

struct Student
{
 int studentID;
 PersonInfo pData;
 short yearInSchool;
 double gpa;
};
```

## Structures as Function Arguments

- May pass members of struct variables to functions:

```
computeGPA(stu.gpa);
```
- May pass entire struct variables to functions:

```
showData(stu);
```
- Can use reference parameter if function needs to modify contents of structure variable



## Excerpts from Program 11-6

```
8 struct InventoryItem
9 {
10 int partNum; // Part number
11 string description; // Item description
12 int onHand; // Units on hand
13 double price; // Unit price
14 };

61 void showItem(InventoryItem p)
62 {
63 cout << fixed << showpoint << setprecision(2);
64 cout << "Part Number: " << p.partNum << endl;
65 cout << "Description: " << p.description << endl;
66 cout << "Units On Hand: " << p.onHand << endl;
67 cout << "Price: $" << p.price << endl;
68 }
```

## Revised showItem Function

```
void showItem(const InventoryItem &p)
{
 cout << fixed << showpoint << setprecision(2);
 cout << "Part Number: " << p.partNum << endl;
 cout << "Description: " << p.description << endl;
 cout << "Units On Hand: " << p.onHand << endl;
 cout << "Price: $" << p.price << endl;
}
```

## Returning a Structure from a Function - Example

```
Student getStudentData()
{
 Student tempStu;
 cin >> tempStu.studentID;
 getline(cin, tempStu.pData.name);
 getline(cin, tempStu.pData.address);
 getline(cin, tempStu.pData.city);
 cin >> tempStu.yearInSchool;
 cin >> tempStu.gpa;
 return tempStu;
}
```

```
25
26 // Get data about the circle.
27 c = getInfo();
28
29 // Calculate the circle's area.
30 c.area = PI * pow(c.radius, 2.0);
31
32 // Display the circle data.
33 cout << "The radius and area of the circle are:\n";
34 cout << fixed << setprecision(2);
35 cout << "Radius: " << c.radius << endl;
36 cout << "Area: " << c.area << endl;
37 return 0;
38 }
39
```

## Structures as Function Arguments - Notes

- Using value parameter for structure can slow down a program, waste space
- Using a reference parameter will speed up program, but function may change data in structure
- Using a `const` reference parameter allows read-only access to reference parameter, does not waste space, speed

## Returning a Structure from a Function

- Function can return a `struct`:  
Student getStudentData(); // prototype  
stu1 = getStudentData(); // call
- Function must define a local structure
  - for internal use
  - for use with `return` statement

### Program 11-7

```
1 // This program uses a function to return a structure. This
2 // is a modification of Program 11-2.
3 #include <iostream>
4 #include <iomanip>
5 #include <cmath> // For the pow function
6 using namespace std;
7
8 // Constant for pi.
9 const double PI = 3.14159;
10
11 // Structure declaration
12 struct Circle
13 {
14 double radius; // A circle's radius
15 double diameter; // A circle's diameter
16 double area; // A circle's area
17 };
18
19 // Function prototype
20 Circle getInfo();
21
22 int main()
23 {
24 Circle c; // Define a structure variable
```

```
40 //*****
41 // Definition of function getInfo. This function uses a local
42 // variable, tempCircle, which is a circle structure. The user
43 // enters the diameter of the circle, which is stored in
44 // tempCircle.diameter. The function then calculates the radius
45 // which is stored in tempCircle.radius. tempCircle is then
46 // returned from the function.
47 //*****
48
49 Circle getInfo()
50 {
51 Circle tempCircle; // Temporary structure variable
52
53 // Store circle data in the temporary variable.
54 cout << "Enter the diameter of a circle: ";
55 cin >> tempCircle.diameter;
56 tempCircle.radius = tempCircle.diameter / 2.0;
57
58 // Return the temporary variable.
59 return tempCircle;
60 }
```

### Program Output with Example Input Shown in Bold

```
Enter the diameter of a circle: 10 [Enter]
The radius and area of the circle are:
Radius: 5.00
Area: 78.54
```

## Pointers to Structures

- A structure variable has an address
- Pointers to structures are variables that can hold the address of a structure:  
`Student *stuPtr;`
- Can use `&` operator to assign address:  
`stuPtr = & stu1;`
- Structure pointer can be a function parameter

## Accessing Structure Members via Pointer Variables

- Must use `()` to dereference pointer variable, not field within structure:  
`cout << (*stuPtr).studentID;`
- Can use structure pointer operator to eliminate `()` and use clearer notation:  
`cout << stuPtr->studentID;`

## From Program 11-8

```
42 void getData(Student *s)
43 {
44 // Get the student name.
45 cout << "Student name: ";
46 getline(cin, s->name);
47
48 // Get the student ID number.
49 cout << "Student ID Number: ";
50 cin >> s->idNum;
51
52 // Get the credit hours enrolled.
53 cout << "Credit Hours Enrolled: ";
54 cin >> s->creditHours;
55
56 // Get the GPA.
57 cout << "Current GPA: ";
58 cin >> s->gpa;
59 }
```

## Enumerated Data Types

- An enumerated data type is a programmer-defined data type. It consists of values known as *enumerators*, which represent integer constants.

## Enumerated Data Types

- Example:

```
enum Day { MONDAY, TUESDAY,
 WEDNESDAY, THURSDAY,
 FRIDAY };
```

- The identifiers `MONDAY`, `TUESDAY`, `WEDNESDAY`, `THURSDAY`, and `FRIDAY`, which are listed inside the braces, are *enumerators*. They represent the values that belong to the `Day` data type.

## Enumerated Data Types

```
enum Day { MONDAY, TUESDAY,
 WEDNESDAY, THURSDAY,
 FRIDAY };
```

Note that the enumerators are not strings, so they aren't enclosed in quotes. They are identifiers.

## Enumerated Data Types

- Once you have created an enumerated data type in your program, you can define variables of that type. Example:

```
Day workDay;
```

- This statement defines `workDay` as a variable of the `Day` type.

## Enumerated Data Types

- We may assign any of the enumerators `MONDAY`, `TUESDAY`, `WEDNESDAY`, `THURSDAY`, or `FRIDAY` to a variable of the `Day` type. Example:

```
workDay = WEDNESDAY;
```

# Enumerated Data Types

- So, what is an *enumerator*?
- Think of it as an integer named constant
- Internally, the compiler assigns integer values to the enumerators, beginning at 0.

# Enumerated Data Types

```
enum Day { MONDAY, TUESDAY,
 WEDNESDAY, THURSDAY,
 FRIDAY };
```

In memory...

```
MONDAY = 0
TUESDAY = 1
WEDNESDAY = 2
THURSDAY = 3
FRIDAY = 4
```

## Enumerated Data Types

- Using the `Day` declaration, the following code...

```
cout << MONDAY << " "
 << WEDNESDAY << " "
 << FRIDAY << endl;
```

...will produce this output:

```
0 2 4
```

## Assigning an integer to an `enum` Variable

- You cannot directly assign an integer value to an `enum` variable. This will not work:

```
workDay = 3; // Error!
```

- Instead, you must cast the integer:

```
workDay = static_cast<Day>(3);
```

## Assigning an Enumerator to an `int` Variable

- You CAN assign an enumerator to an `int` variable. For example:

```
int x;
x = THURSDAY;
```

- This code assigns 3 to `x`.

## Comparing Enumerator Values

- Enumerator values can be compared using the relational operators. For example, using the `Day` data type the following code will display the message "Friday is greater than Monday."

```
if (FRIDAY > MONDAY)
{
 cout << "Friday is greater "
 << "than Monday.\n";
}
```

### Program 11-9

```
1 // This program demonstrates an enumerated data type.
2 #include <iostream>
3 #include <iomanip>
4 using namespace std;
5
6 enum Day { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY };
7
8 int main()
9 {
10 const int NUM_DAYS = 5; // The number of days
11 double sales[NUM_DAYS]; // To hold sales for each day
12 double total = 0.0; // Accumulator
13 int index; // Loop counter
14
15 // Get the sales for each day.
16 for (index = MONDAY; index <= FRIDAY; index++)
17 {
18 cout << "Enter the sales for day "
19 << index << ": ";
20 cin >> sales[index];
21 }
22
```

### Program 11-9 (Continued)

```
23 // Calculate the total sales.
24 for (index = MONDAY; index <= FRIDAY; index++)
25 total += sales[index];
26
27 // Display the total.
28 cout << "The total sales are $" << setprecision(2)
29 << fixed << total << endl;
30
31 return 0;
32 }
```

### Program Output with Example Input Shown in Bold

```
Enter the sales for day 0: 1525.00
Enter the sales for day 1: 1896.50
Enter the sales for day 2: 1975.63
Enter the sales for day 3: 1678.33
Enter the sales for day 4: 1498.52
The total sales are $8573.98
```

## Enumerated Data Types

- Program 11-9 shows enumerators used to control a loop:

```
// Get the sales for each day.
for (index = MONDAY; index <= FRIDAY; index++)
{
 cout << "Enter the sales for day "
 << index << ": ";
 cin >> sales[index];
}
```

## Using Math Operators with `enum` Variables

- You can run into problems when trying to perform math operations with `enum` variables. For example:

```
Day day1, day2; // Define two Day variables.
day1 = TUESDAY; // Assign TUESDAY to day1.
day2 = day1 + 1; // ERROR! Will not work!
```

- The third statement will not work because the expression `day1 + 1` results in the integer value 2, and you cannot store an int in an `enum` variable.

## Anonymous Enumerated Types

- An *anonymous enumerated type* is simply one that does not have a name. For example, in Program 11-10 we could have declared the enumerated type as:

```
enum { MONDAY, TUESDAY,
 WEDNESDAY, THURSDAY,
 FRIDAY };
```

## Using Math Operators with `enum` Variables

- You can fix this by using a cast to explicitly convert the result to `Day`, as shown here:

```
// This will work.
day2 = static_cast<Day>(day1 + 1);
```

## Using an `enum` Variable to Step through an Array's Elements

- Because enumerators are stored in memory as integers, you can use them as array subscripts. For example:

```
enum Day { MONDAY, TUESDAY, WEDNESDAY,
 THURSDAY, FRIDAY };
const int NUM_DAYS = 5;
double sales[NUM_DAYS];
sales[MONDAY] = 1525.0;
sales[TUESDAY] = 1896.5;
sales[WEDNESDAY] = 1975.63;
sales[THURSDAY] = 1678.33;
sales[FRIDAY] = 1498.52;
```

## Using an `enum` Variable to Step through an Array's Elements

- Remember, though, you cannot use the `++` operator on an `enum` variable. So, the following loop will NOT work.

```
Day workDay; // Define a Day variable
// ERROR!!! This code will NOT work.
for (workDay = MONDAY; workDay <= FRIDAY; workDay++)
{
 cout << "Enter the sales for day "
 << workDay << ": ";
 cin >> sales[workDay];
}
```

## Using an `enum` Variable to Step through an Array's Elements

- You must rewrite the loop's update expression using a cast instead of `++`:

```
for (workDay = MONDAY; workDay <= FRIDAY;
 workDay = static_cast<Day>(workDay + 1))
{
 cout << "Enter the sales for day "
 << workDay << ": ";
 cin >> sales[workDay];
}
```

### Program 11-10

```
1 // This program demonstrates an enumerated data type.
2 #include <iostream>
3 #include <iomanip>
4 using namespace std;
5
6 enum Day { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY };
7
8 int main()
9 {
10 const int NUM_DAYS = 5; // The number of days
11 double sales[NUM_DAYS]; // To hold sales for each day
12 double total = 0.0; // Accumulator
13 Day workDay; // Loop counter
14
15 // Get the sales for each day.
16 for (workDay = MONDAY; workDay <= FRIDAY;
17 workDay = static_cast<Day>(workDay + 1))
18 {
19 cout << "Enter the sales for day "
20 << workDay << ": ";
21 cin >> sales[workDay];
22 }
```



## Enumerators Must Be Unique Within the same Scope

- Enumerators must be unique within the same scope. (Unless strongly typed)
- For example, an error will result if both of the following enumerated types are declared within the same scope:

```
enum Presidents { MCKINLEY, ROOSEVELT, TAFT };

enum VicePresidents { ROOSEVELT, FAIRBANKS,
 SHERMAN };
```

ROOSEVELT is declared twice.

```
23 // Calculate the total sales.
24 for (workDay = MONDAY; workDay <= FRIDAY;
25 workDay = static_cast<Day>(workDay + 1))
26 total += sales[workDay];
27
28 // Display the total.
29 cout << "The total sales are $" << setprecision(2)
30 << fixed << total << endl;
31
32 return 0;
33
34 }
```

### Program Output with Example Input Shown in Bold

```
Enter the sales for day 0: 1525.00
Enter the sales for day 1: 1896.50
Enter the sales for day 2: 1975.33
Enter the sales for day 3: 1678.33
Enter the sales for day 4: 1498.52
The total sales are $8573.98
```

## Using Strongly Typed `enums` in C++ 11

- In C++ 11, you can use a new type of `enum`, known as a *strongly typed enum*
- Allows you to have multiple enumerators in the same scope with the same name

```
enum class Presidents { MCKINLEY, ROOSEVELT, TAFT };
enum class VicePresidents { ROOSEVELT, FAIRBANKS, SHERMAN };
```

- Prefix the enumerator with the name of the `enum`, followed by the `::` operator:  
Presidents prez = Presidents::ROOSEVELT;  
VicePresidents vp = VicePresidents::ROOSEVELT;
- Use a cast operator to retrieve integer value:

```
int x = static_cast<int>(Presidents::ROOSEVELT);
```

## Declaring the Type and Defining the Variables in One Statement

- You can declare an enumerated data type and define one or more variables of the type in the same statement. For example:

```
enum Car { PORSCHE, FERRARI, JAGUAR } sportsCar;
```

This code declares the `Car` data type and defines a variable named `sportsCar`.

## Procedural and Object-Oriented Programming

- Procedural programming focuses on the process/actions that occur in a program
- Object-Oriented programming is based on the data and the functions that operate on it. Objects are instances of ADTs (Abstract Data Types) that represent the data and its functions

## Limitations of Procedural Programming

- If the data structures change, many functions must also be changed
- Programs that are based on complex function hierarchies are:
  - difficult to understand and maintain
  - difficult to modify and extend
  - easy to break

## Object-Oriented Programming Terminology

- class: like a `struct` (allows bundling of related variables), but variables and functions in the class can have different properties than in a `struct`
- object: an instance of a `class`, in the same way that a variable can be an instance of a `struct`

## Object-Oriented Programming Terminology

- attributes: members of a class
- methods or behaviors: member functions of a class

## More on Objects

- **data hiding**: restricting access to certain members of an object
- **public interface**: members of an object that are available outside of the object. This allows the object to provide access to some data and functions without sharing its internal details and design, and provides some protection from data corruption

## Introduction to Classes

- Objects are created from a `class`
- Format:

```
class ClassName
{
 declaration;
 declaration;
};
```

## Class Example

```
class Rectangle
{
 private:
 double width;
 double length;
 public:
 void setWidth(double);
 void setLength(double);
 double getWidth() const;
 double getLength() const;
 double getArea() const;
};
```

## Access Specifiers

- Used to control access to members of the class
- **public**: can be accessed by functions outside of the class
- **private**: can only be called by or accessed by functions that are members of the class

## More on Access Specifiers

- Can be listed in any order in a class
- Can appear multiple times in a class
- If not specified, the default is `private`

## Using `const` With Member Functions

- **const** appearing after the parentheses in a member function declaration specifies that the function will not change any data in the calling object.

```
double getWidth() const;
double getLength() const;
double getArea() const;
```

## Defining a Member Function

- When defining a member function:
  - Put prototype in class declaration
  - Define function using class name and scope resolution operator (`::`)

```
int Rectangle::setWidth(double w)
{
 width = w;
}
```

## Accessors and Mutators

- **Mutator**: a member function that stores a value in a private member variable, or changes its value in some way
- **Accessor**: function that retrieves a value from a private member variable. Accessors do not change an object's data, so they should be marked `const`.

# Defining an Instance of a Class

- An object is an instance of a class
- Defined like structure variables:  
Rectangle r;
- Access members using dot operator:  
r.setWidth(5.2);  
cout << r.getWidth();
- Compiler error if attempt to access private member using dot operator

Program 13-1

```
1 // This program demonstrates a simple class.
2 #include <iostream>
3 using namespace std;
4
5 // Rectangle class declaration.
6 class Rectangle
7 {
8 private:
9 double width;
10 double length;
11 public:
12 void setWidth(double);
13 void setLength(double);
14 double getWidth() const;
15 double getLength() const;
16 double getArea() const;
17 };
18
19 //*****
20 // setWidth assigns a value to the width member. *
21 //*****
22
23 void Rectangle::setWidth(double w)
24 {
25 width = w;
26 }
27
28 //*****
29 // setLength assigns a value to the length member. *
30 //*****
31
```

Program 13-1 (Continued)

```
32 void Rectangle::setLength(double len)
33 {
34 length = len;
35 }
36
37 //*****
38 // getWidth returns the value in the width member. *
39 //*****
40
41 double Rectangle::getWidth() const
42 {
43 return width;
44 }
45
46 //*****
47 // getLength returns the value in the length member. *
48 //*****
49
50 double Rectangle::getLength() const
51 {
52 return length;
53 }
54
```

Program 13-1 (Continued)

```
55 //*****
56 // getArea returns the product of width times length. *
57 //*****
58
59 double Rectangle::getArea() const
60 {
61 return width * length;
62 }
63
64 //*****
65 // Function main
66 //*****
67
68 int main()
69 {
70 Rectangle box; // Define an instance of the Rectangle class
71 double rectWidth; // local variable for width
72 double rectLength; // local variable for length
73
74 // Get the rectangle's width and length from the user.
75 cout << "This program will calculate the area of a";
76 cout << "rectangle. What is the width? ";
77 cin >> rectWidth;
78 cout << "What is the length? ";
79 cin >> rectLength;
80
81 // Store the width and length of the rectangle
82 // in the box object.
83 box.setWidth(rectWidth);
84 box.setLength(rectLength);
85
```

Program 13-1 (Continued)

```
86
87 // Display the rectangle's data.
88 cout << "Here is the rectangle's data:\n";
89 cout << "Width: " << box.getWidth() << endl;
90 cout << "Length: " << box.getLength() << endl;
91 cout << "Area: " << box.getArea() << endl;
92 return 0;
93 }
```

## Program Output

```
This program will calculate the area of a
rectangle. What is the width? 10 [Enter]
What is the length? 5 [Enter]
Here is the rectangle's data:
Width: 10
Length: 5
Area: 50
```

## Avoiding Stale Data

- Some data is the result of a calculation.
- In the Rectangle class the area of a rectangle is calculated.
  - length x width
- If we were to use an area variable here in the Rectangle class, its value would be dependent on the length and the width.
- If we change length or width without updating area, then area would become *stale*.
- To avoid stale data, it is best to calculate the value of that data within a member function rather than store it in a variable.

## Pointer to an Object

- Can define a pointer to an object:  
Rectangle \*rPtr = nullptr;
- Can access public members via pointer:  
rPtr = &otherRectangle;  
rPtr->setLength(12.5);  
cout << rPtr->getLength() << endl;

## Dynamically Allocating an Object

- We can also use a pointer to dynamically allocate an object.
- ```
1 // Define a Rectangle pointer.
2 Rectangle *rectPtr = nullptr;
3
4 // Dynamically allocate a Rectangle object.
5 rectPtr = new Rectangle;
6
7 // Store values in the object's width and length.
8 rectPtr->setWidth(10.0);
9 rectPtr->setLength(15.0);
10
11 // Delete the object from memory.
12 delete rectPtr;
13 rectPtr = nullptr;
```


Why Have Private Members?

- Making data members `private` provides data protection
- Data can be accessed only through `public` functions
- Public functions define the class's public interface

Separating Specification from Implementation

- Place class declaration in a header file that serves as the class specification file. Name the file `ClassName.h`, for example, `Rectangle.h`
- Place member function definitions in `ClassName.cpp`, for example, `Rectangle.cpp`. File should `#include` the class specification file
- Programs that use the class must `#include` the class specification file, and be compiled and linked with the member function definitions

Inline Member Functions

- Member functions can be defined
 - inline: in class declaration
 - after the class declaration
- Inline appropriate for short function bodies:

```
int getWidth() const
{ return width; }
```

Rectangle Class with Inline Member Functions

```
1 // Specification file for the Rectangle class
2 // This version uses some inline member functions.
3 #ifndef RECTANGLE_H
4 #define RECTANGLE_H
5
6 class Rectangle
7 {
8     private:
9         double width;
10        double length;
11    public:
12        void setWidth(double);
13        void setLength(double);
14        double getWidth() const
15            { return width; }
16        double getLength() const
17            { return length; }
18        double getArea() const
19            { return width * length; }
20 };
21 #endif
```

Tradeoffs – Inline vs. Regular Member Functions

- Regular functions – when called, compiler stores return address of call, allocates memory for local variables, etc.
- Code for an inline function is copied into program in place of call – larger executable program, but no function call overhead, hence faster execution

Constructors

- Member function that is automatically called when an object is created
- Purpose is to construct an object
- Constructor function name is class name
- Has no return type

Contents of Rectangle.h (Version 3)

```
1 // Specification file for the Rectangle class
2 // This version has a constructor.
3 #ifndef RECTANGLE_H
4 #define RECTANGLE_H
5
6 class Rectangle
7 {
8     private:
9         double width;
10        double length;
11    public:
12        Rectangle(); // Constructor
13        void setWidth(double);
14        void setLength(double);
15        double getWidth() const
16            { return width; }
17        double getLength() const
18            { return length; }
19        double getArea() const
20            { return width * length; }
21 };
22 #endif
```

Contents of Rectangle.cpp (Version 3)

```
1 // Implementation file for the Rectangle class.
2 // This version has a constructor.
3 #include "Rectangle.h" // Needed for the Rectangle class
4 #include <iostream> // Needed for cout
5 #include <cstdlib> // Needed for the exit function
6 using namespace std;
7
8 //*****
9 // The constructor initializes width and length to 0.0. *
10 //*****
11
12 Rectangle::Rectangle()
13 {
14     width = 0.0;
15     length = 0.0;
16 }
```

Continues...


```

17
18 //*****
19 // setWidth sets the value of the member variable width. *
20 //*****
21
22 void Rectangle::setWidth(double w)
23 {
24     if (w >= 0)
25         width = w;
26     else
27     {
28         cout << "Invalid width!\n";
29         exit(EXIT_FAILURE);
30     }
31 }
32 //*****
33 // setLength sets the value of the member variable length. *
34 //*****
35
36 void Rectangle::setLength(double len)
37 {
38     if (len >= 0)
39         length = len;
40     else
41     {
42         cout << "Invalid length!\n";
43         exit(EXIT_FAILURE);
44     }
45 }

```

(continued)

Program 13-7

```

1 // This program uses the Rectangle class's constructor.
2 #include <iostream>
3 #include "Rectangle.h" // Needed for Rectangle class
4 using namespace std;
5
6 int main()
7 {
8     Rectangle box; // Define an instance of the Rectangle class
9
10    // Display the rectangle's data.
11    cout << "Here is the rectangle's data:\n";
12    cout << "Width: " << box.getWidth() << endl;
13    cout << "Length: " << box.getLength() << endl;
14    cout << "Area: " << box.getArea() << endl;
15    return 0;
16 }

```

Program Output

```

Here is the rectangle's data:
Width: 0
Length: 0
Area: 0

```

In-Place Initialization

- If you are using C++11 or later, you can initialize a member variable in its declaration statement, just as you can with a regular variable.
- This is known as in-place initialization. Here is an example:

```

class Rectangle
{
private:
    double width = 0.0;
    double length = 0.0;
public:
    Public member functions appear here...
};

```

Default Constructors

- A default constructor is a constructor that takes no arguments.
- If you write a class with no constructor at all, C++ will write a default constructor for you, one that does nothing.
- A simple instantiation of a class (with no arguments) calls the default constructor:

```
Rectangle r;
```

Passing Arguments to Constructors

- To create a constructor that takes arguments:
 - indicate parameters in prototype:

```
Rectangle(double, double);
```

- Use parameters in the definition:

```

Rectangle::Rectangle(double w, double len)
{
    width = w;
    length = len;
}

```

Passing Arguments to Constructors

- You can pass arguments to the constructor when you create an object:

```
Rectangle r(10, 5);
```

More About Default Constructors

- If all of a constructor's parameters have default arguments, then it is a default constructor. For example:

```
Rectangle(double = 0, double = 0);
```

- Creating an object and passing no arguments will cause this constructor to execute:

```
Rectangle r;
```

Classes with No Default Constructor

- When all of a class's constructors require arguments, then the class has NO default constructor.
- When this is the case, you must pass the required arguments to the constructor when creating an object.

Destructors

- Member function automatically called when an object is destroyed
- Destructor name is ~classname, e.g., ~Rectangle
- Has no return type; takes no arguments
- Only one destructor per class, i.e., it cannot be overloaded
- If constructor allocates dynamic memory, destructor should release it

Contents of InventoryItem.h (Version 1)

```
1 // Specification file for the InventoryItem class.
2 #ifndef INVENTORYITEM_H
3 #define INVENTORYITEM_H
4 #include <cstring> // Needed for strlen and strcpy
5
6 // InventoryItem class declaration.
7 class InventoryItem
8 {
9 private:
10     char *description; // The item description
11     double cost;        // The item cost
12     int units;          // Number of units on hand
```

Contents of InventoryItem.h Version 1

```
13 public:
14     // Constructor
15     InventoryItem(char *desc, double c, int u)
16     { // Allocate just enough memory for the description.
17         description = new char (strlen(desc) + 1);
18
19         // Copy the description to the allocated memory.
20         strcpy(description, desc);
21
22         // Assign values to cost and units.
23         cost = c;
24         units = u;
25
26     // Destructor
27     ~InventoryItem()
28     { delete [] description; }
29
30     const char *getDescription() const
31     { return description; }
32
33     double getCost() const
34     { return cost; }
35
36     int getUnits() const
37     { return units; }
38 };
39 #endif
```

(continued)

Program 13-12

```
1 // This program demonstrates a class with a destructor.
2 #include <iostream>
3 #include "ContactInfo.h"
4 using namespace std;
5
6 int main()
7 {
8     // Define a ContactInfo object with the following data:
9     // Name: Kristen Lee Phone Number: 555-2021
10    ContactInfo entry("Kristen Lee", "555-2021");
11
12    // Display the object's data.
13    cout << "Name: " << entry.getName() << endl;
14    cout << "Phone Number: " << entry.getPhoneNumber() << endl;
15    return 0;
16 }
```

Program Output

Name: Kristen Lee
Phone Number: 555-2021

Constructors, Destructors, and Dynamically Allocated Objects

- When an object is dynamically allocated with the new operator, its constructor executes:

```
Rectangle *r = new Rectangle(10, 20);
```

- When the object is destroyed, its destructor executes:

```
delete r;
```

Overloading Constructors

- A class can have more than one constructor
- Overloaded constructors in a class must have different parameter lists:

```
Rectangle();
Rectangle(double);
Rectangle(double, double);
```

```
1 // This class has overloaded constructors.
2 #ifndef INVENTORYITEM_H
3 #define INVENTORYITEM_H
4 #include <string>
5 using namespace std;
6
7 class InventoryItem
8 {
9 private:
10     string description; // The item description
11     double cost;        // The item cost
12     int units;          // Number of units on hand
13 public:
14     // Constructor #1
15     InventoryItem()
16     { // Initialize description, cost, and units.
17         description = "";
18         cost = 0.0;
19         units = 0; }
20
21     // Constructor #2
22     InventoryItem(string desc)
23     { // Assign the value to description.
24         description = desc;
25
26         // Initialize cost and units.
27         cost = 0.0;
28         units = 0; }
```

Continues...

```
29
30     // Constructor #3
31     InventoryItem(string desc, double c, int u)
32     { // Assign values to description, cost, and units.
33         description = desc;
34         cost = c;
35         units = u; }
36
37     // Mutator functions
38     void setDescription(string d)
39     { description = d; }
40
41     void setCost(double c)
42     { cost = c; }
43
44     void setUnits(int u)
45     { units = u; }
46
47     // Accessor functions
48     string getDescription() const
49     { return description; }
50
51     double getCost() const
52     { return cost; }
53
54     int getUnits() const
55     { return units; }
56 };
57 #endif
```

Constructor Delegation

- Sometimes a class will have multiple constructors that perform a similar set of steps. For example, look at the following `Contact` class:

```

class Contact
{
private:
    string name;
    string email;
    string phone;
public:
    // Constructor #1 (default)
    Contact()
    {
        name = "";
        email = "";
        phone = "";
    }

    // Constructor #2
    Contact(string n, string e, string p)
    {
        name = n;
        email = e;
        phone = p;
    }

    // Other member functions follow...
};

```

Constructor Delegation

- Both constructors perform a similar operation: They assign values to the name, email, and phone member variables.
- The default constructor assigns empty strings to the members, and the parameterized constructor assigns specified values to the members.

```
class Contact
{
private:
    string name;
    string email;
    string phone;
public:
    // Constructor #1 (default)
    Contact() {}
    { name = "";
      email = "";
      phone = "";
    }
    // Constructor #2
    Contact(string n, string e, string p)
    { name = n;
      email = e;
      phone = p;
    }
    Other member functions follow...
};
```

Constructor Delegation

- In C++ 11, it is possible for one constructor to call another constructor in the same class.
- This is known as *constructor delegation*.

```
class Contact
{
private:
    string name;
    string email;
    string phone;
public:
    // Constructor #1 (default)
    Contact() : Contact("", "", "")
    {
    }

    // Constructor #2
    Contact(string n, string e, string p)
    {
        name = n;
        email = e;
        phone = p;
    }

    Other member functions follow...
};
```

Only One Default Constructor and One Destructor

- Do not provide more than one default constructor for a class: one that takes no arguments and one that has default arguments for all parameters

```
Square();  
Square(int = 0); // will not compile
```
- Since a destructor takes no arguments, there can only be one destructor for a class

Using Private Member Functions

- A `private` member function can only be called by another member function
- It is used for internal processing by the class, not for use outside of the class
- See the `createDescription` function in `ContactInfo.h` (Version 2)

Arrays of Objects

- Objects can be the elements of an array:

```
InventoryItem inventory[40];
```
- Default constructor for object is used when array is defined

Arrays of Objects

- Must use initializer list to invoke constructor that takes arguments:

```
InventoryItem inventory[3] =
    { "Hammer", "Wrench", "Pliers" };
```

Arrays of Objects

- If the constructor requires more than one argument, the initializer must take the form of a function call:

[illegible]

Arrays of Objects

- It isn't necessary to call the same constructor for each object in an array:

```
InventoryItem inventory[3] = { "Hammer",  
    InventoryItem("Wrench", 8.75, 20),  
    "Pliers" };
```

Accessing Objects in an Array

- Objects in an array are referenced using subscripts
- Member functions are referenced using dot notation:

```
inventory[2].setUnits(30);  
cout << inventory[2].getUnits();
```

Program 13-14

```
1 // This program demonstrates an array of class objects.  
2 #include <iostream>  
3 #include <iomanip>  
4 #include "InventoryItem.h"  
5 using namespace std;  
6  
7 int main()  
8 {  
9     const int NUM_ITEMS = 5;  
10    InventoryItem inventory[NUM_ITEMS] = {  
11        InventoryItem("Hammer", 6.95, 12),  
12        InventoryItem("Wrench", 8.75, 20),  
13        InventoryItem("Pliers", 3.75, 10),  
14        InventoryItem("Ratchet", 7.95, 14),  
15        InventoryItem("Screwdriver", 2.50, 22) };  
16
```

Program 13-14 (Continued)

```
17     cout << setw(14) << "Inventory Item"  
18         << setw(8) << "Cost" << setw(8)  
19         << setw(16) << "Units on Hand\n";  
20     cout << "-----\n";  
21  
22     for (int i = 0; i < NUM_ITEMS; i++)  
23     {  
24         cout << setw(14) << inventory[i].getDescription();  
25         cout << setw(8) << inventory[i].getCost();  
26         cout << setw(7) << inventory[i].getUnits() << endl;  
27     }  
28  
29     return 0;  
30 }
```

Program Output

Inventory Item	Cost	Units on Hand
Hammer	6.95	12
Wrench	8.75	20
Pliers	3.75	10
Ratchet	7.95	14
Screwdriver	2.5	22

The Unified Modeling Language

- UML stands for *Unified Modeling Language*.
- The UML provides a set of standard diagrams for graphically depicting object-oriented systems

UML Class Diagram

- A UML diagram for a class has three main sections.



Example: A Rectangle Class

```
class Rectangle  
{  
    private:  
        double width;  
        double length;  
    public:  
        bool setWidth(double);  
        bool setLength(double);  
        double getWidth() const;  
        double getLength() const;  
        double getArea() const;  
};
```

Rectangle
width length
setWidth() setLength() getWidth() getLength() getArea()

UML Access Specification Notation

- In UML you indicate a private member with a minus (-) and a public member with a plus(+).

These member variables are private.

These member functions are public.

Rectangle
- width - length
+ setWidth() + setLength() + getWidth() + getLength() + getArea()

UML Data Type Notation

- To indicate the data type of a member variable, place a colon followed by the name of the data type after the name of the variable.

```
- width : double
- length : double
```

UML Parameter Type Notation

- To indicate the data type of a function's parameter variable, place a colon followed by the name of the data type after the name of the variable.

```
+ setWidth(w : double)
```

UML Function Return Type Notation

- To indicate the data type of a function's return value, place a colon followed by the name of the data type after the function's parameter list.

```
+ setWidth(w : double) : void
```

The Rectangle Class

Rectangle
- width : double - length : double
+ setWidth(w : double) : bool + setLength(len : double) : bool + getWidth() : double + getLength() : double + getArea() : double

Showing Constructors and Destructors

No return type listed for constructors or destructors

Constructors →

Destructor →

InventoryItem
- description : char* - cost : double - units : int - createDescription(size : int, value : char*) : void
+ InventoryItem() : + InventoryItem(desc : char*) : + InventoryItem(desc : char*, c : double, u : int) : + ~InventoryItem() : + setDescription(d : char*) : void + setCost(c : double) : void + setUnits(u : int) : void + getDescription() : char* + getCost() : double + getUnits() : int